Clustering algorithms

Prepared by: Radwa Taher

K-means

K-means is an unsupervised machine learning clustering algorithm that partitions a dataset into **K distinct groups (clusters)** by assigning each data point to the nearest cluster center (centroid) and iteratively updating the centroids until the clusters become stable.

K-means (numeric example)

Given Data (1D Example)

We have the following points:

```
1, 2, 8, 9
```

Assume we want:

```
K = 2
```

So we want to divide the data into **2 clusters**.

K-means (numeric example)

Step 1: Choose Initial Centroids

K-means starts by selecting random centroids.

Point	Distance to C1 (1)	Distance to C2 (8)	Assigned Cluster
1	0	7	C1
2	1	6	C1
8	7	0	C2
9	8	1	C2

Assume we randomly choose:

C1 = 1, C2 = 8

Step 3: Form Clusters

Cluster 1 → {1, 2}

Cluster 2 → {8, 9}

K-means (numeric example)

Step 4: Update Centroids

New centroid = mean of points inside the cluster.

Cluster 1:

(1+2)/2=1.5(1 + 2) / 2 = 1.5(1+2)/2=1.5

New C1 = 1.5

Cluster 2:

(8+9)/2=8.5(8 + 9) / 2 = 8.5(8+9)/2=8.5

New C2 = 8.5

K-means (numeric example)

Step 5: Repeat Assignment

Now calculate distances again using new centroids.

The clusters do not change. ✅

Point	To 1.5	To 8.5	Assigned
1	0.5	7.5	C1
2	0.5	6.5	C1
8	6.5	0.5	C2
9	7.5	0.5	C2

🎉 Algorithm Stops

Final clusters: Cluster 1 → {1, 2}
Cluster 2 → {8, 9}

Final centroids: 1.5 and 8.5

K means practical.


```
df = pd.read_csv("driver-data.csv", index_col="id")
df.head()
```

id: Unique Id of the driver
mean_dist_day: Mean distance driven by driver per day
mean_over_speed_perc: Mean percentage of time a driver was > 5 mph over the speed limit

id	mean_dist_day	mean_over_speed_perc
3423311935	71.24	28
3423313212	52.53	25
3423313724	64.54	27
3423311373	55.69	22
3423310999	54.58	25

K means practical.

- **What does it represent?**

After K-Means:

1. Assigns points to clusters
2. Recalculates the mean of each cluster
3. Repeats until convergence

The final means of the clusters are saved in:

```
kmeans.cluster_centers_
```

```
from sklearn.cluster import KMeans
```

```
kmeans = KMeans(n_clusters=4)
```

```
kmeans.fit(df)
```

```
KMeans
KMeans(n_clusters=4)
```

```
kmeans.cluster_centers_
```

```
array([[ 41.03274371,   5.38757862],
       [180.09647059,  18.30788486],
       [ 57.44532765,   5.29161206],
       [ 50.65456576,  33.0719603 ]])
```

K means practical.

```
import numpy as np
```

```
unique, counts = np.unique(kmeans.labels_, return_counts=True)
print (unique,counts)
```

```
[0 1 2 3] [1564  695  104 1637]
```

[10]
✓ 0s

```
df["cluster"] = kmeans.labels_
df["cluster"]
```

...

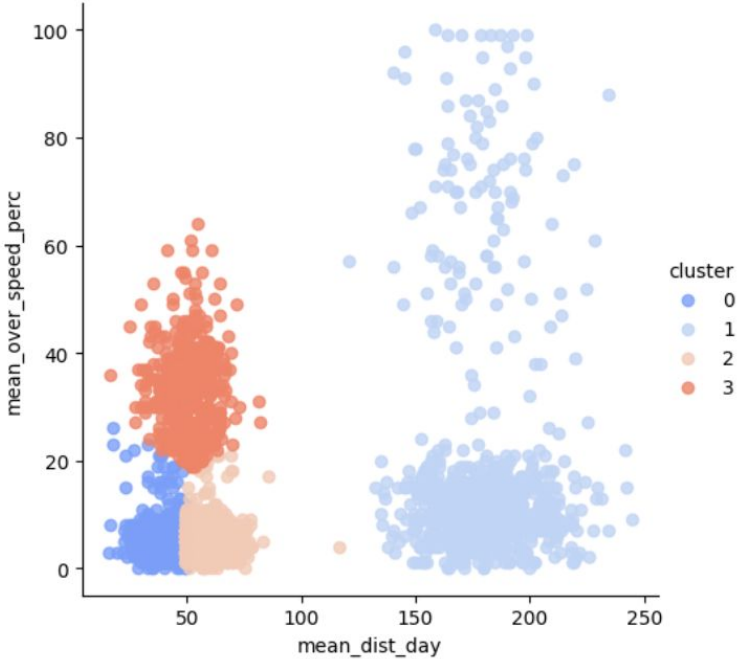
id	cluster
3423311935	0
3423313212	0
3423313724	0
3423311373	0
3423310999	0

4000 rows × 1 columns

dtype: int32

K means practical.

```
sns.lmplot(x = 'mean_dist_day', y = 'mean_over_speed_perc',
           data = df, hue='cluster', palette='coolwarm',
           fit_reg = False)
```



K means practical

```
kmeans.inertia_
```

```
785728.1080029513
```

Mathematically:

$$\text{Inertia} = \sum_{i=1}^{n} \|x_i - \mu_{cluster(i)}\|^2$$

Where:

- x_i = data point
- μ = centroid of the cluster
- The distance is squared (Euclidean distance squared)

K means practical

What does "inertia" mean?

It measures:

The total squared distance between each data point and its assigned cluster centroid.

What does it tell us?

- **Low inertia** → Points are close to their centroids → Tight clusters
- **High inertia** → Points are far from centroids → Loose clusters

---

K-means tries to **minimize inertia**.

K-Means(Iris dataset)

```
# Standard imports
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import dendrogram, linkage
```

K-Means(Iris dataset)

```
# Load Iris dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names
df = pd.DataFrame(X, columns=feature_names)
df['target'] = y
df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	ta
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	

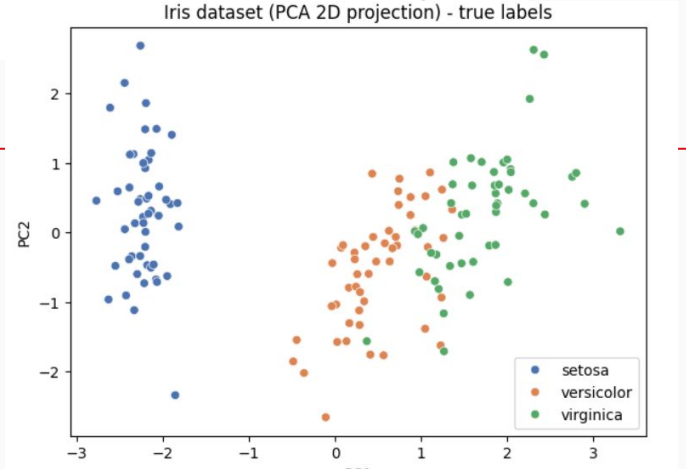
K-Means(Iris dataset)

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pd.DataFrame(X_scaled, columns=feature_names).head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	-0.900681	1.019004	-1.340227	-1.315444
1	-1.143017	-0.131979	-1.340227	-1.315444
2	-1.385353	0.328414	-1.397064	-1.315444
3	-1.506521	0.098217	-1.283389	-1.315444
4	-1.021849	1.249201	-1.340227	-1.315444

K-Means(Iris dataset)

```
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)
plt.figure(figsize=(7,5))
sns.scatterplot(x=X_pca[:,0], y=X_pca[:,1], hue=iris.target_names[y], palette='deep')
plt.title('Iris dataset (PCA 2D projection) - true labels')
plt.xlabel('PC1'); plt.ylabel('PC2')
plt.show()
```



K-Means (Iris dataset)

```
# Elbow method (inertia) & silhouette for k=2..6
inertias = []
sil_scores = []
K = range(2,15)
for k in K:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_scaled)
    inertias.append(km.inertia_)
    sil_scores.append(silhouette_score(X_scaled, labels))
```

K-Means (Iris dataset)

What is the Silhouette Score?

The **Silhouette Score** is a metric used to evaluate how good your clustering is.

It answers this question:

> Is each data point placed in the correct cluster?

It measures:

- ✅ How close a point is to its own cluster
- ❌ How far it is from other clusters

K-Means (Iris dataset)

✅ Silhouette Score Equation

For one data point i:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Where:

- $a(i)$ = average distance between point i and other points in the **same cluster** (cluster cohesion)
- $b(i)$ = average distance between point i and points in the **nearest other cluster** (cluster separation)

K-Means (Iris dataset)

✅ Numeric Example

Suppose we have 2 clusters:

Cluster 1:

$$A,\ B,\ C$$

Cluster 2:

$$D,\ E,\ F$$

We want to compute the silhouette score for point **A**.

K-Means (Iris dataset)

Step 1: Compute $a(A)$

Assume distances from **A** to points in its own cluster:

- dist(A,B) = 2
- dist(A,C) = 4

So:

$$a(A) = \frac{2+4}{2} = 3$$

So point A is on average **3 units** away from its own cluster.

K-Means (Iris dataset)

Step 2: Compute $b(A)$

Now compute distance from **A** to points in the nearest other cluster (Cluster 2):

Assume:

- dist(A,D) = 8
- dist(A,E) = 9
- dist(A,F) = 7

Average distance:

$$b(A) = \frac{8 + 9 + 7}{3} = 8$$

So point A is **8 units** away from the closest other cluster.

K-Means (Iris dataset)

Step 3: Apply the Silhouette Formula

$$s(A) = \frac{b(A) - a(A)}{\max(a(A), b(A))}$$

Substitute values:

$$s(A) = \frac{8 - 3}{\max(3, 8)}$$

$$s(A) = \frac{5}{8} = 0.625$$

✅ Final Result

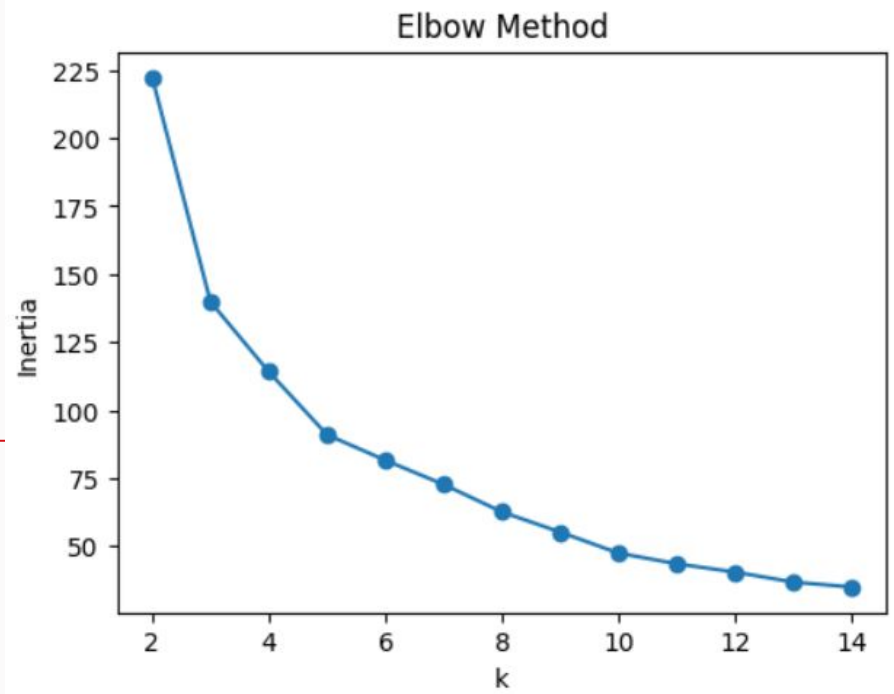
$$s(A) = 0.625$$

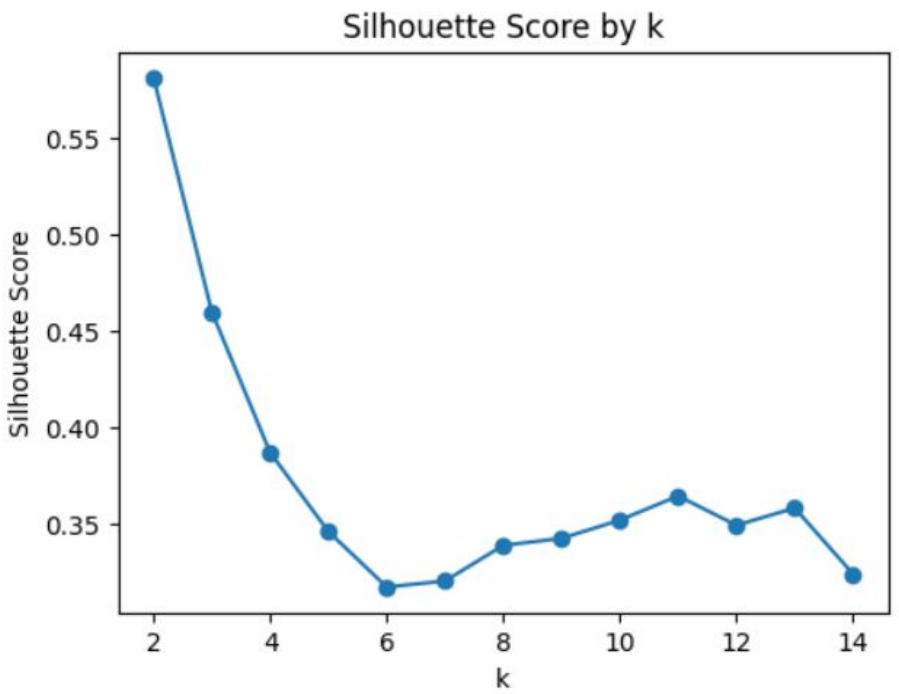
K-Means (Iris dataset)

```
plt.figure(figsize=(12,4))
plt.subplot(1,2,1)
plt.plot(K, inertias, '-o')
plt.xlabel('k'); plt.ylabel('Inertia'); plt.title('Elbow Method')

plt.subplot(1,2,2)
plt.plot(K, sil_scores, '-o')
plt.xlabel('k'); plt.ylabel('Silhouette Score'); plt.title('Silhouette Score by k')
plt.show()
```

K-Means (Iris dataset)





K-Means (Iris dataset)

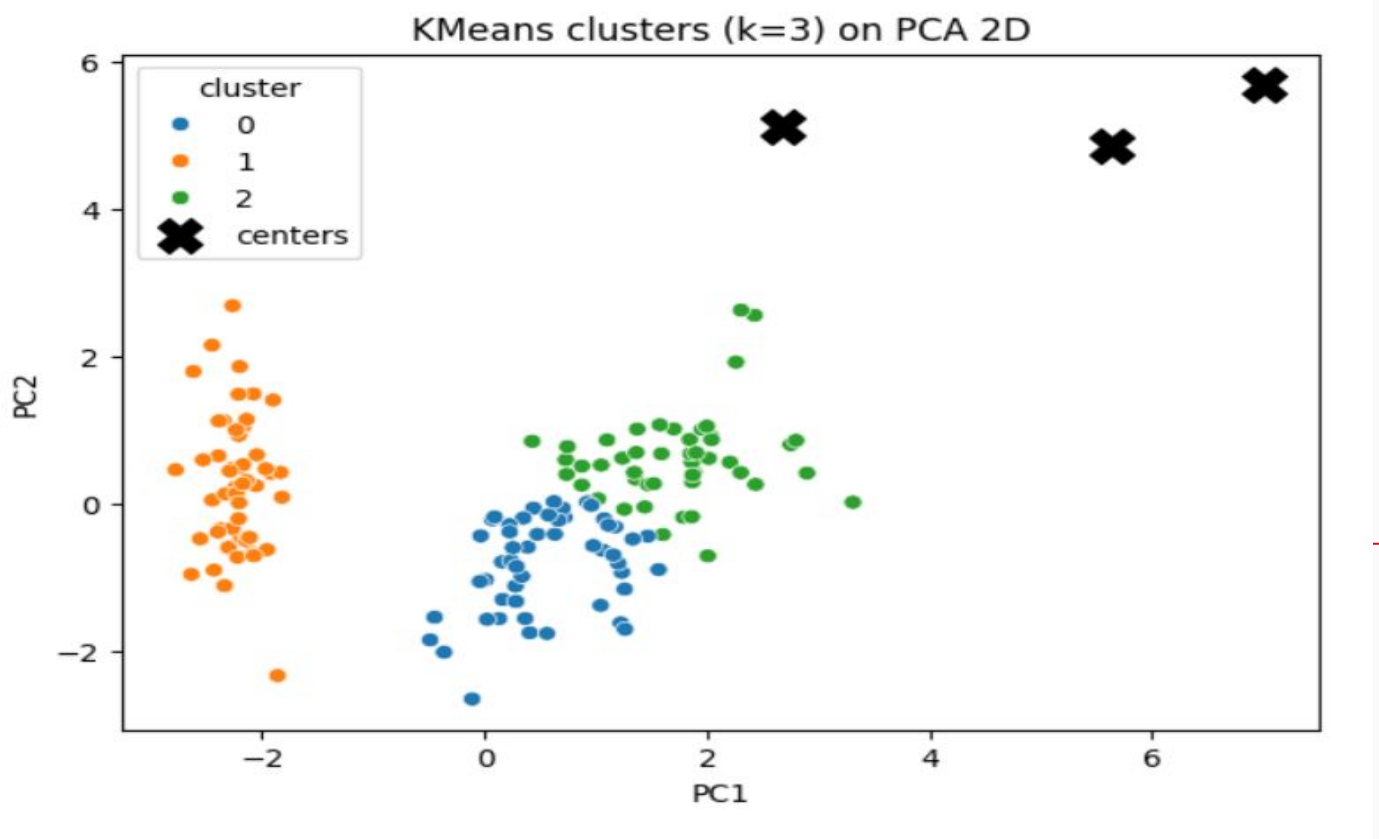
```
# Choose k=3 (known for Iris)
k = 3
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
klabels = kmeans.fit_predict(X_scaled)
print('Silhouette score (k=3):', round(silhouette_score(X_scaled, klabels),3))
```

```
Silhouette score (k=3): 0.46
```

K-Means (Iris dataset)

```
# Visualize KMeans clusters on PCA space
plt.figure(figsize=(7,5))
sns.scatterplot(x=X_pca[:,0], y=X_pca[:,1], hue=klabels, palette='tab10', legend='full')
centers = pca.transform(scaler.inverse_transform(kmeans.cluster_centers_))  # project cluster centers to PCA space
plt.scatter(centers[:,0], centers[:,1], c='black', s=200, marker='X', label='centers')
plt.title(f'KMeans clusters (k={k}) on PCA 2D'); plt.xlabel('PC1'); plt.ylabel('PC2')
plt.legend(title='cluster')
plt.show()
```

K-Means (Iris dataset)



Agglomerative Clusters

🔷 What Are Agglomerative Clusters?

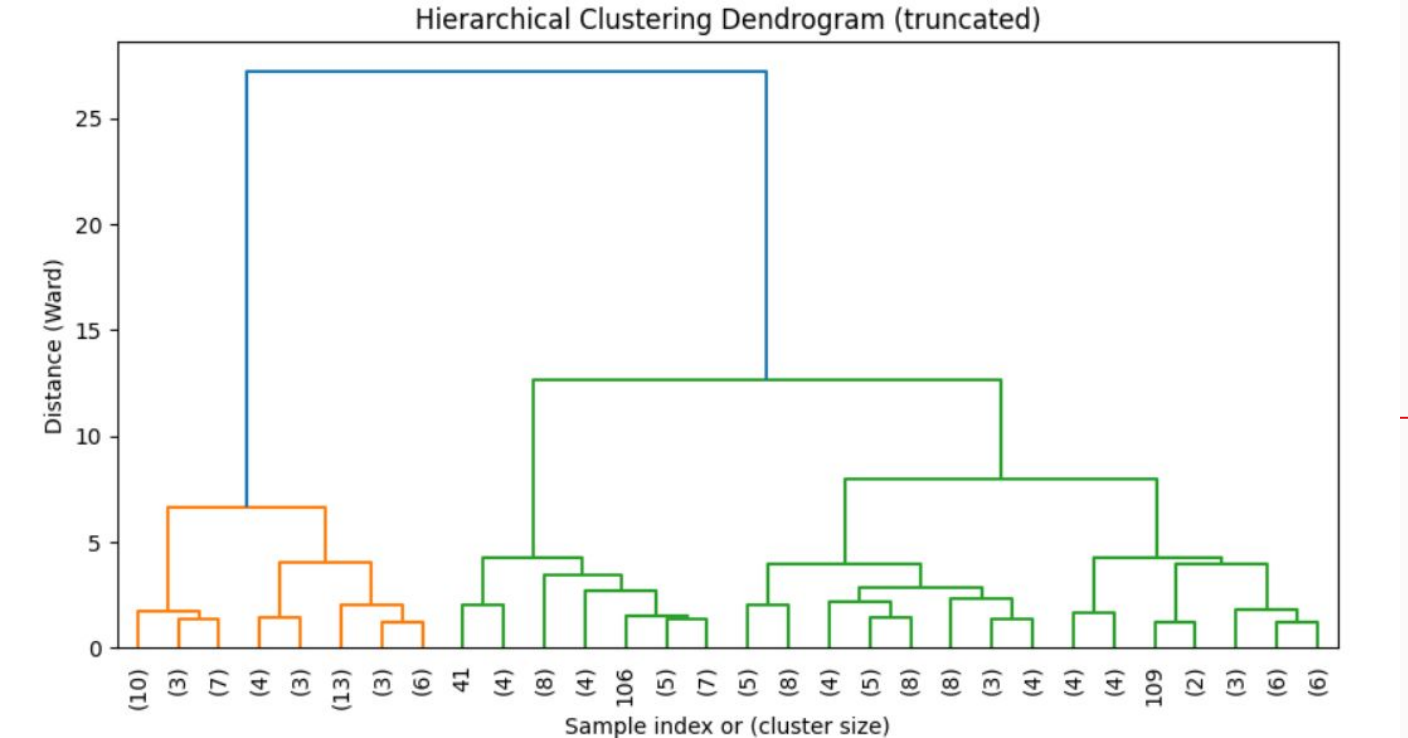
Agglomerative clusters are the groups formed using **agglomerative hierarchical clustering.**

This method follows a:

Bottom-up approach

It starts with each data point as its own cluster and then repeatedly merges the closest clusters until the desired number of clusters is reached.

Agglomerative Clusters



Agglomerative Clusters

We have the following data points: $1,\ 2,\ 3,\ 6,\ 10,\ 11$

We name them:

- A = 1, B = 2, C = 3, D = 6 ← a middle point, E = 10, F = 11

Initial Step

Each point starts as its own cluster: $\{1\}, \{2\}, \{3\}, \{6\}, \{10\}, \{11\}$

Agglomerative Clusters

✅ First Merge (All Linkage Methods Agree)

The closest points are

- 1 and 2 → distance = 1
- 2 and 3 → distance = 1
- 10 and 11 → distance = 1

So after merging, we get:

$$\{1, 2, 3\}, \{6\}, \{10, 11\}$$

Agglomerative Clusters

⭐ 1️⃣ Single Linkage

Single linkage depends on:

> The closest pair of points between two clusters

Distance between {1,2,3} and {6}

The closest point is 3:

$$|3 - 6| = 3$$

Distance between {6} and {10,11}

The closest point is 10:

$$|6 - 10| = 4$$

Decision:

$$3 < 4$$

$$\{1, 2, 3, 6\}, \{10, 11\}$$

⭐ 2️⃣ Complete Linkage

Complete linkage depends on:

> The farthest pair of points between two clusters

Distance between {1,2,3} and {6}

The farthest point is 1:

$$|1 - 6| = 5$$

Distance between {6} and {10,11}

The farthest point is 11:

$$|6 - 11| = 5$$

Decision:

Both distances are equal:

$$5 = 5$$

Agglomerative Clusters

⭐ 3️⃣ Average Linkage

Average linkage depends on:

> The average distance between all pairs of points

Between {1,2,3} and {6}

Distances:

$$|1-6| = 5,\ |2-6| = 4,\ |3-6| = 3$$

Average:

$$(5+4+3)/3 = 4$$

Between {6} and {10,11}

Distances:

$$|6-10| = 4,\ |6-11| = 5$$

Average:

$$(4+5)/2 = 4.5$$

Decision:

$$4 < 4.5$$

So Average linkage merges:

$$\{1,2,3\} + \{6\}$$

Agglomerative Clusters

⭐ 4️⃣ Ward Linkage

Ward linkage depends on:

> The smallest increase in variance (cluster spread)

Merge {1,2,3} with {6}

New cluster:

$$\{1, 2, 3, 6\}$$

Mean:

$$(1 + 2 + 3 + 6)/4 = 3$$

Variance (sum of squared deviations):

$$(1 - 3)^2 + (2 - 3)^2 + (3 - 3)^2 + (6 - 3)^2$$

$$= 4 + 1 + 0 + 9 = 14$$

Agglomerative Clusters

1️⃣ Single Linkage (Nearest Neighbor)

When to use it:

✔ When you want to detect **connected or chain-like structures**
✔ Useful in cases where clusters may not be compact but rather stretched

Example applications:

- Finding clusters in **geographic or network-like data**

2️⃣ Complete Linkage (Farthest Neighbor)

When to use it:

✔ When you want **compact, tight clusters**
✔ When you want to avoid chaining

Example applications:

- Customer segmentation where groups must be clearly separated

Agglomerative Clusters

✅3️⃣Average Linkage (Mean Distance)

Example applications:

- Biological data clustering (genes, proteins)
- Text/document clustering

✅4️⃣Ward Linkage (Variance Minimization)

When to use it:

✔ Best for **numeric continuous data**
✔ When you want clusters similar to K-Means
✔ Produces **compact and well-separated clusters**

Example applications:

- Driver behavior clustering
- Any feature-based ML dataset

Agglomerative Clusters

```
# Linkage (Ward) on scaled data for dendrogram
linked = linkage(X_scaled, method='ward')

plt.figure(figsize=(10,5))
dendrogram(linked, truncate_mode='lastp', p=30, leaf_rotation=90., leaf_font_size=10.)
plt.title('Hierarchical Clustering Dendrogram (truncated)')
plt.xlabel('Sample index or (cluster size)')
plt.ylabel('Distance (Ward)')
plt.show()
```

Agglomerative Clusters

```
# Fit AgglomerativeClustering with n_clusters=3
agg = AgglomerativeClustering(n_clusters=3, linkage='ward')
alabels = agg.fit_predict(X_scaled)
print('Silhouette score (agglomerative, n=3):', round(silhouette_score(X_scaled, alabels),3))
```

```
Silhouette score (agglomerative, n=3): 0.447
```